# Indexing

## What is Indexing?

Indexing is a technique used in databases to optimize queries and improve the performance of database operations. It involves creating a data structure, known as an index, that allows for faster data retrieval based on specific columns or fields.

In a database, an index is essentially a pointer to a set of rows in a table that share a common value or set of values in one or more columns. The index is stored separately from the table, which makes it faster to search for specific data.

When a query is executed on a table, the database engine can use the index to quickly locate the relevant rows instead of having to scan the entire table. This can significantly reduce the time it takes to perform queries and other database operations.

## Explain and Analyze A Query

An execution plan (or query plan) is the sequence of steps that the database plans to take to execute a query. One can check the execution plan of a query using the "EXPLAIN" keyword.
Syntax:-
EXPLAIN [options] <Query>;

One can understand the time that each query has taken using the "ANALYZE" Option.
Syntax:-
EXPLAIN ANALYZE <QUERY>;

Eg 1:-

```
explain analyze select * from film where release_year = 2006;
```

| | QUERY PLAN text |
|---|---|
| 1 | Seq Scan on film (cost=0.00..66.50 rows=1000 width=384) (actual time=0.008..0.171 rows=1000 loops=1) |
| 2 | Filter: ((release_year)::integer = 2006) |
| 3 | Planning Time: 0.071 ms |
| 4 | Execution Time: 0.197 ms |

Here the Query plan states that it will perform a sequential scan of all the rows, and filter the rows using the condition release_year = 2006.

Eg 2:-

```
explain analyze select * from film where film_id = 1;
```

| | QUERY PLAN text |
|---|---|
| 1 | Index Scan using film_pkey on film (cost=0.28..8.29 rows=1 width=384) (actual time=0.071..0.072 rows=1 loops=1) |
| 2 | Index Cond: (film_id = 1) |
| 3 | Planning Time: 0.132 ms |
| 4 | Execution Time: 0.086 ms |

Here the query is such that it requires a searching operation on a particular attribute. It utilizes the index built on the primary key, to conduct the search.
**Every table has an index on the primary key & Unique Constraint by default.**
Reference:-
https://www.postgresql.org/docs/current/sql-explain.html

## B-Tree

In B-tree indexing, data is organized into a tree-like structure that consists of nodes, each of which contains a range of keys and pointers to child nodes. B-Tree index is used by default when type of indexing is not specified.

General Syntax for creating an Index:-

```
CREATE INDEX index_name ON table_name [type]
(
        column_name [ASC | DESC] [NULLS {FIRST | LAST }],
        .....
);
```

Eg:-

```
CREATE INDEX idx_address_phone ON address(phone);
```

OR

```
CREATE INDEX idx_address_phone ON address USING BTREE(phone);
```

Once the Index has been created, any query requiring the phone column to be searched will utilize the index.

Eg:-

```
EXPLAIN ANALYZE SELECT *
FROM address
WHERE phone = '223664661973';
```

**Query without the Index**

| | QUERY PLAN text |
|---|---|
| 1 | Seq Scan on address (cost=0.00..15.54 rows=1 width=61) (actual time=0.027..0.103 rows=1 loops=1) |
| 2 | Filter: ((phone)::text = '223664661973'::text) |
| 3 | Rows Removed by Filter: 602 |
| 4 | Planning Time: 0.117 ms |
| 5 | Execution Time: 0.127 ms |

**Query with the Index**

| | QUERY PLAN text |
|---|---|
| 1 | Index Scan using idx_address_phone on address (cost=0.28..8.29 rows=1 width=61) (actual time=0.068..0.069 rows=1 loops=1) |
| 2 | Index Cond: ((phone)::text = '223664661973'::text) |
| 3 | Planning Time: 0.994 ms |
| 4 | Execution Time: 0.082 ms |

The important thing to note is the time taken to execute the query, Queries utilizing the index, will in most cases outperform Queries, not utilizing the index.

Indexes are created for certain scenarios, B-Tree indexes are created when a query involves the following:-

- **Range searches:** For example, finding all the records in a database table with a value between a minimum and maximum value.
- **Equality matches:** For example, finding all the records in a database table with a specific value in a particular column.
- **Prefix matches:** For example, finding all the records in a database table where a particular column starts with a specific string.

More about B-tree:-

https://www.cs.cornell.edu/courses/cs3110/2012sp/recitations/rec25-B-trees/rec25.html

## Drop an Index

Syntax:-

DROP INDEX <index_name>;

Eg:-

```
create index some_idx on customer(customer_id);

drop index some_idx;
```

## Hash Index

For a huge database structure, it can be almost next to impossible to search all the index values through all its levels and then reach the destination data block to retrieve the desired data. Hashing is an effective technique to calculate the direct location of a data record on the disk without using index structure.
Hashing uses hash functions with **search keys as parameters** to generate the address of a data record.

**General Syntax for creating a Hash Index:**

CREATE HASH INDEX <index_name> ON <table_name> USING HASH(<column_name>)

Example:
Notice the execution time without any indexing;

```
5  explain analyse select * from address where phone = '635297277345' ;
```

Data Output | Messages | Notifications

| | QUERY PLAN text |
|---|---|
| 1 | Seq Scan on address (cost=0.00..15.54 rows=1 width=61) (actual time=0.021..0.163 rows=1 loops=1) |
| 2 | Filter: ((phone)::text = '635297277345'::text) |
| 3 | Rows Removed by Filter: 602 |
| 4 | Planning Time: 0.086 ms |
| 5 | Execution Time: 0.202 ms |

Creating a hash index;

```
1  create index phone_col_index on address using HASH(phone);
```

Analyzing a select query that is using the hash index created;

```
2  explain analyse select * from address where phone = '635297277345';
```

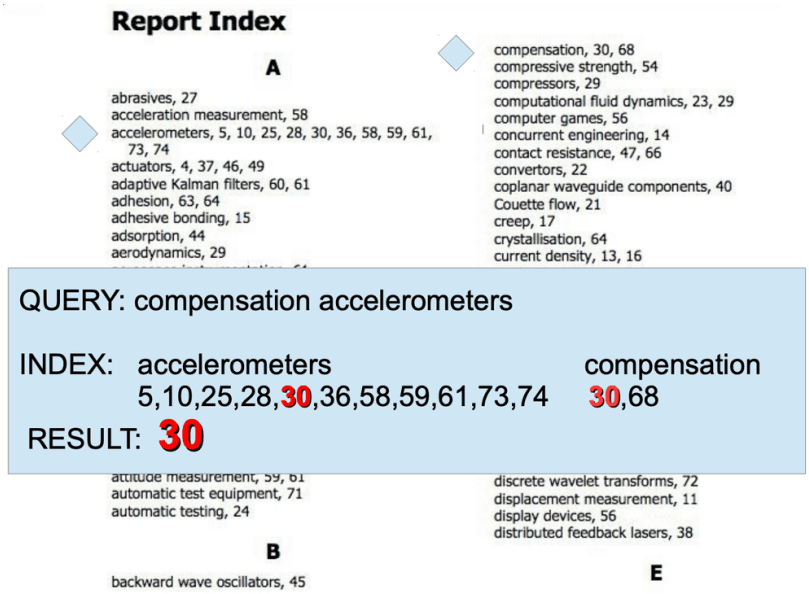
Data Output | Messages | Notifications

| | QUERY PLAN text |
|---|---|
| 1 | Index Scan using phone_col_index on address (cost=0.00..8.02 rows=1 width=61) (actual time=0.036..0.037 rows=1 loop… |
| 2 | Index Cond: ((phone)::text = '635297277345'::text) |
| 3 | Planning Time: 0.091 ms |
| 4 | Execution Time: 0.062 ms |

Notice that one can use B-Tree indexing for range searches but Hash Indexing is limited to only equality constraints. (i.e. WHERE phone = '635297277345'). One cannot use the Hash Index for range searches. Nevertheless , hash Indexing is faster than B-Tree in exact searches as it directly computes the memory location of the data.

## GIN (Generalized Inverted iNdex) Index

The GIN index type was designed to deal with data types that are subdividable and you want to search for individual component values (array elements, lexemes in a text document, etc). These cannot be Indexed efficiently by B-Trees, hence the need for GIN index.



GIN index is like the table of contents in a book, where the heap pointers (to the actual table) are the page numbers. Multiple entries can be combined to yield a specific result, like the search for “compensation accelerometers” in the above example.

One use case of utilizing GIN is to index the ILIKE/LIKE operation.

In order to index for LIKE/ILIKE we need to load the trigram extension,

```
create extension pg_trgm;
```

More about the Trigram Extension:-

https://www.postgresql.org/docs/current/pgtrgm.html

Trigram Indexing works as so, if you have the word "postgres" in a text column, the trigrams for that word would be "pos", "ost", "stg", "gre", "res". If you then perform a search for the term "post", the trigrams "pos" and "ost" match, and the trigram index can quickly find all rows that contain these trigrams. This makes trigram indexes very useful for text search operations that require fuzzy matching, such as substring matching, searching for similar-sounding names or misspelled words.

Eg:-

```
CREATE INDEX trgm_idx ON customer USING gin (email gin_trgm_ops);
```

Above creates a GIN index for the email column of the customer table. gin_trgm_ops is an opclass i.e operators to be used by the index for that column, here we are using trigrams.

Consider the query:-

```
explain analyze select * from customer where email ilike '%123%';
```

Output:-

| | QUERY PLAN<br>text |
|---|---|
| 1 | Bitmap Heap Scan on customer (cost=12.00..16.01 rows=1 width=70) (actual time=0.011..0.011 rows=0 loops=1) |
| 2 | Recheck Cond: ((email)::text ~~* '%123%'::text) |
| 3 | -> Bitmap Index Scan on trgm_idx (cost=0.00..12.00 rows=1 width=0) (actual time=0.009..0.009 rows=0 loops=1) |
| 4 | Index Cond: ((email)::text ~~* '%123%'::text) |
| 5 | Planning Time: 0.281 ms |
| 6 | Execution Time: 0.038 ms |

NOTE:- GIN indexes use Bitmap Index Scans, hence they show up as Bitmap heap scans.

Consider the following Query:-

```
explain analyze select * from customer where email ilike '%@%';
```

| | QUERY PLAN<br>text |
|---|---|
| 1 | Seq Scan on customer (cost=0.00..16.49 rows=599 width=70) (actual time=0.013..0.787 rows=599 loops=1) |
| 2 | Filter: ((email)::text ~~* '%@%'::text) |
| 3 | Planning Time: 0.258 ms |
| 4 | Execution Time: 0.813 ms |

Above doesn't utilize the GIN index as the pattern appears in nearly every email, therefore the query engine will perform a sequential scan rather than going through the overhead of using an Index.

There are other operations possible with GIN, reference:-

https://www.postgresql.org/docs/current/gin-builtin-opclasses.html

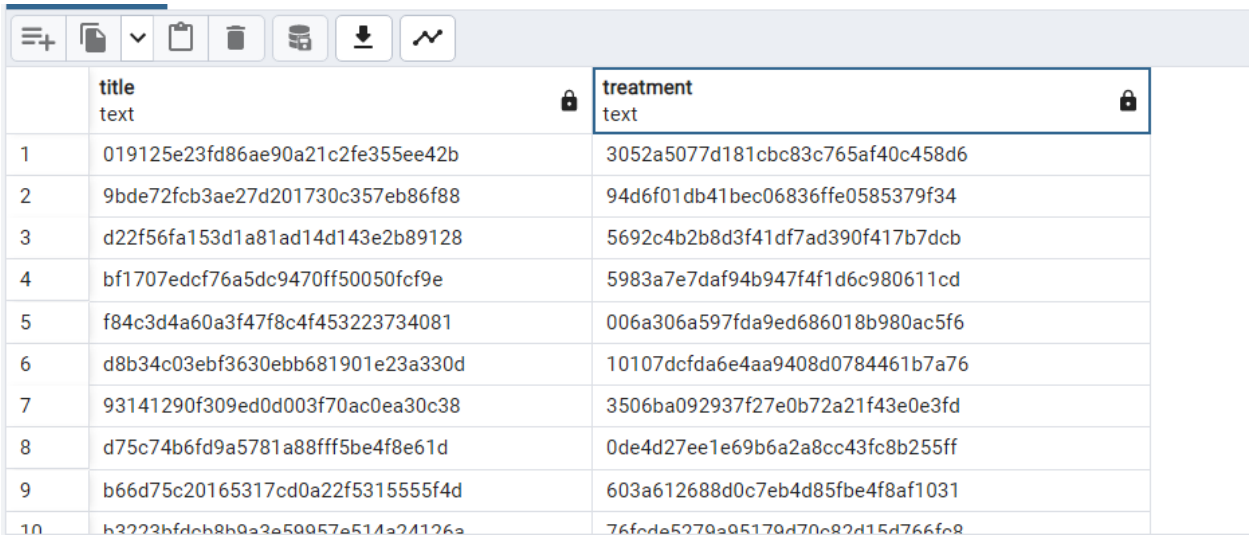
**Eg:- Using randomly generated text data**

```
CREATE TABLE sample(
title text,
treatment text
);

INSERT INTO sample(title, treatment)
select md5(random()::text), md5(random()::text)
from (select * from generate_series(1,10000) as id) as x;
```

- MD5() function calculates the MD5 hash of a string and returns the result in hexadecimal, the return type is in TEXT.
- random() will generate a random number between 0 and 1. We are then typecasting the result to text.
- generate_series (1,10000) will return 10000 rows with the value starting from 1 to 10,000.

Table contents:-



| | title<br>text | treatment<br>text |
|---|---|---|
| 1 | 019125e23fd86ae90a21c2fe355ee42b | 3052a5077d181cbc83c765af40c458d6 |
| 2 | 9bde72fcb3ae27d201730c357eb86f88 | 94d6f01db41bec06836ffe0585379f34 |
| 3 | d22f56fa153d1a81ad14d143e2b89128 | 5692c4b2b8d3f41df7ad390f417b7dcb |
| 4 | bf1707edcf76a5dc9470ff50050fcf9e | 5983a7e7daf94b947f4f1d6c980611cd |
| 5 | f84c3d4a60a3f47f8c4f453223734081 | 006a306a597fda9ed686018b980ac5f6 |
| 6 | d8b34c03ebf3630ebb681901e23a330d | 10107dcfda6e4aa9408d0784461b7a76 |
| 7 | 93141290f309ed0d003f70ac0ea30c38 | 3506ba092937f27e0b72a21f43e0e3fd |
| 8 | d75c74b6fd9a5781a88fff5be4f8e61d | 0de4d27ee1e69b6a2a8cc43fc8b255ff |
| 9 | b66d75c20165317cd0a22f5315555f4d | 603a612688d0c7eb4d85fbe4f8af1031 |
| 10 | b3223bfdcb8b9a3e59957e514a24126a | 76fcde5279a95179d70c82d15d766fc8 |

Consider that we have query:-

```
explain analyze select * from sample where title ilike '%eca%';
```

The execution plan for the above query is:-

| | QUERY PLAN<br>text |
|---|---|
| 1 | Seq Scan on sample (cost=0.00..249.00 rows=101 width=66) (actual time=0.158..14.321 rows=65 loops=1) |
| 2 | Filter: (title ~~* '%eca%'::text) |
| 3 | Rows Removed by Filter: 9935 |
| 4 | Planning Time: 0.928 ms |
| 5 | Execution Time: 14.353 ms |

We can create a GIN for the above table as so:-

```
create index gin_sample on sample using GIN(title gin_trgm_ops);
```

The Query plan after creating the index:-

| | QUERY PLAN<br>text |
|---|---|
| 1 | Bitmap Heap Scan on sample (cost=12.78..137.45 rows=101 width=66) (actual time=0.024..0.157 rows... |
| 2 | Recheck Cond: (title ~~* '%eca%'::text) |
| 3 | Heap Blocks: exact=53 |
| 4 | -> Bitmap Index Scan on gin_sample (cost=0.00..12.76 rows=101 width=0) (actual time=0.014..0.015 ro... |
| 5 | Index Cond: (title ~~* '%eca%'::text) |
| 6 | Planning Time: 1.542 ms |
| 7 | Execution Time: 0.179 ms |

## Indexing and Order-By

In addition to simply finding the rows to be returned by a query, an index may be able to deliver them in a specific sorted order. This allows a query's ORDER BY specification to be honored without a separate sorting step. **Of the index types currently supported by PostgreSQL, only**

**B-tree can produce sorted output — the other index types return matching rows in an unspecified, implementation-dependent order.**

**By default, B-tree indexes store their entries in ascending order with nulls last (table TID is treated as a tiebreaker column among otherwise equal entries).** This means that a forward scan of an index on column x produces output satisfying ORDER BY x (or more verbosely, ORDER BY x ASC NULLS LAST). The index can also be scanned backward, producing output satisfying ORDER BY x DESC (or more verbosely, ORDER BY x DESC NULLS FIRST, since NULLS FIRST is the default for ORDER BY DESC).

```
explain analyze select * from film order by film_id desc;
```

| | QUERY PLAN text |
|---|---|
| 1 | Index Scan Backward using film_pkey on film (cost=0.28..92.92 rows=1000 width=384) (actual time=1.734..2.371 rows=1000 loops=1) |
| 2 | Planning Time: 1.317 ms |
| 3 | Execution Time: 2.422 ms |

You can adjust the ordering of a B-tree index by including the options ASC, DESC, NULLS FIRST, and/or NULLS LAST when creating the index; for example:

**CREATE INDEX test2_info_nulls_low ON test2 (info NULLS FIRST);**

**CREATE INDEX test3_desc_index ON test3 (id DESC NULLS LAST);**

An index stored in ascending order with nulls first can satisfy either ORDER BY x ASC NULLS FIRST or ORDER BY x DESC NULLS LAST depending on which direction it is scanned in.